Trade-offs

how to justify any design choice

Watch the 3-min video

In one line: every design choice improves one quality and costs another; a senior answer names the choice, the reason from the requirements, what you accept, and how you limit it.

1 There is no free lunch

The same tensions come up in almost every design. Knowing them by name lets you spot them early and talk about them clearly.

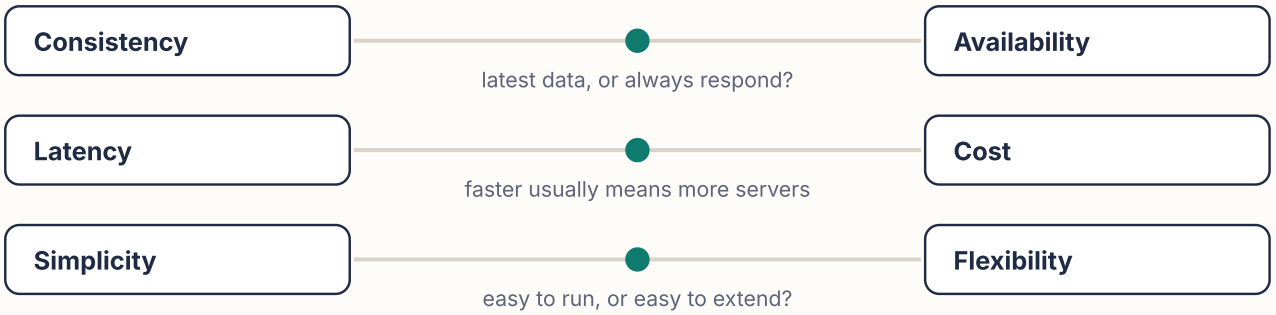


Figure 1. Three trade-offs you will meet in nearly every interview. The requirements decide where the dot sits.

2 A worked example

PRODUCT PAGE AND A CACHE

A product page reads from the database on every request, and the database is struggling. You add a cache with a 60-second expiry. Database load drops by ~80%. The price: a user might see a price up to a minute old.

Now say it out loud with a four-part template:

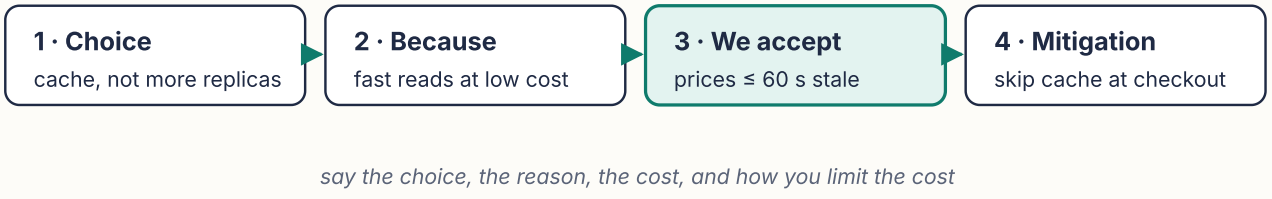


Figure 2. The four-part trade-off statement. Part 3 is what interviewers listen for: you know what you gave up.

"We chose a cache over more read replicas, because the requirement is fast reads at low cost. We accept prices up to 60 seconds stale, and we bypass the cache at checkout, where the price must be exact."

3 From “it depends” to a decision

“It depends” is a fine start but not an answer. Say **what** it depends on, then commit:

1. List the quality attributes that matter (latency, consistency, cost ...).
2. Rank them using the requirements.
3. Score each option against them.
4. Pick, and state what would change your mind.

Option	Read latency	Freshness	Cost	Verdict
Cache (60 s TTL)	~1 ms	≤ 60 s stale	Low	✓ for product pages
More read replicas	~5–10 ms	Replication lag, usually < 1 s	Higher	If freshness matters more
Bigger primary DB	~5–10 ms	Exact	Highest, has a ceiling	Short-term relief

This is the idea behind ATAM-style reasoning (Architecture Trade-off Analysis Method): make the attributes explicit and score options against them.

4 Justifying eventual consistency

Ask what happens if a user sees slightly old data. For a like counter or a feed, a few seconds of delay is harmless, and you gain availability and lower latency. For a bank balance it is not. Add guardrails where needed, like **read-your-writes**: a user always sees their own changes immediately, even if others see them a moment later.

TALKING ABOUT A REAL PROJECT (STAR-STYLE)

Context → **options** you considered → **choice and its cost** → **result with a number**. “We cached repeated LLM answers, cut model cost by 40%, and skipped the cache for time-sensitive questions.”

COMMON MISTAKES

- Presenting a choice as having no downside.
- Stopping at “it depends”.
- Naming a technology instead of the property you need from it.
- One consistency level for every feature.

5 Key terms

Trade-off: gain one quality, pay in another

Eventual consistency: replicas converge, reads may lag

ATAM: scoring options against ranked attributes

Quality attribute: latency, availability, cost ...

Read-your-writes: you always see your own changes

Mitigation: how you limit the cost you accepted

6 Interview questions

> **Q1. How do you justify a design choice?** BASIC
> Name the choice, the requirement that drives it, what you give up, and how you limit that cost. Then say what would make you choose differently.

Q2. When is eventual consistency acceptable? BASIC

When stale data is harmless for that feature (likes, feeds, view counts), and you gain availability or latency. Add read-your-writes so users see their own changes.

Q3. What's wrong with answering “it depends”? BASIC

Nothing as a start, but it isn't an answer. Say what it depends on (the ranked requirements), score the options, and commit.

Q4. Describe a real trade-off from your work. SCENARIO

Use context, options, choice and cost, result with a number. For example: cached repeated LLM answers, cut cost by 40%, accepted stale answers for 1 hour, bypassed the cache for time-sensitive queries.

Q5. Cache or read replicas for a read-heavy product catalogue? SCENARIO

A cache with a short TTL if slightly stale data is fine: cheaper and ~1 ms reads. Read replicas if data must be fresher or queries are too varied to cache. Often both: replicas behind a cache.

Q6. Your team wants microservices for a 3-person startup. How do you respond? SCENARIO

Weigh flexibility against operational cost. With 3 engineers, a modular monolith is usually faster to build and run; split services later where scaling or team boundaries demand it.

Q7. A like counter shows 1,203 to one user and 1,207 to another. Is that a bug? TRICKY

Not if the requirement allows eventual consistency for likes. It becomes a bug only if the product needs exact counts (e.g. billing by likes), then you need strongly consistent counting.

Q8. How do you know which trade-off the interviewer wants you to make? TRICKY

You don't guess: you derive it from the requirements you agreed on, state it, and offer the alternative. Interviewers score the reasoning, not a specific pick.

Next: SD-A-006 · Vertical Scaling backendroadmap.com